```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
data = pd.read_csv('WineQT.csv')
data.head()
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	densit
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.997
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.996
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.997
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.998
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.997

Next steps: Generate code with data | New interactive sheet

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1143 entries, 0 to 1142
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   fixed acidity         1143 non-null   float64
 1   volatile acidity      1143 non-null   float64
 2   citric acid           1143 non-null   float64
 3   residual sugar        1143 non-null   float64
 4   chlorides             1143 non-null   float64
 5   free sulfur dioxide   1143 non-null   float64
 6   total sulfur dioxide  1143 non-null   float64
 7   density               1143 non-null   float64
 8   pH                    1143 non-null   float64
 9   sulphates             1143 non-null   float64
 10  alcohol               1143 non-null   float64
 11  quality               1143 non-null   int64  
 12  Id                    1143 non-null   int64  
dtypes: float64(11), int64(2)
memory usage: 116.2 KB
```

```
data.isnull().sum()
```

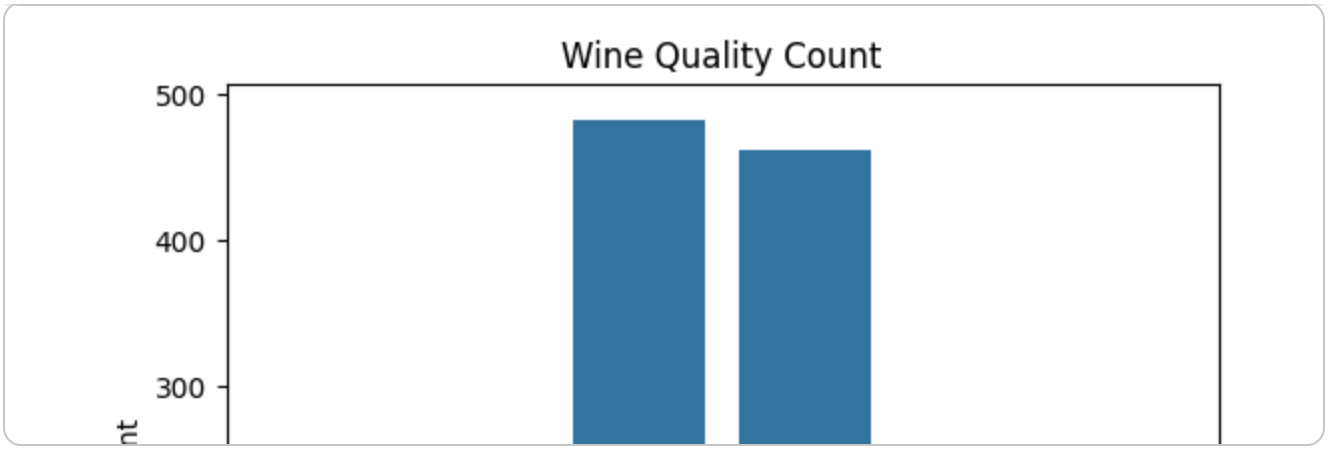
	0
fixed acidity	0
volatile acidity	0
citric acid	0
residual sugar	0
chlorides	0
free sulfur dioxide	0
total sulfur dioxide	0
density	0
pH	0
sulphates	0
alcohol	0
quality	0
Id	0

dtype: int64

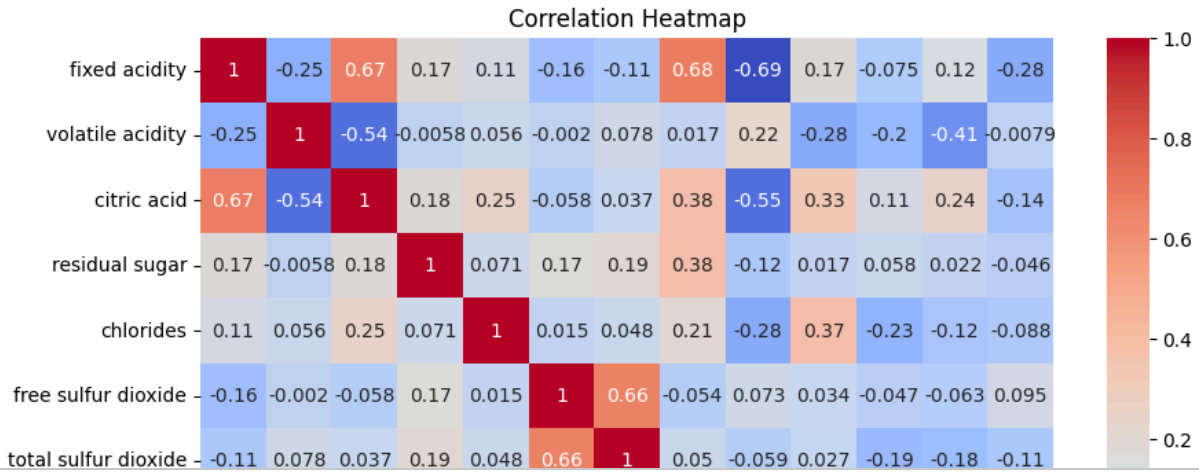
```
data.describe()
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	fre sulfu dioxid
count	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.00000
mean	8.311111	0.531339	0.268364	2.532152	0.086933	15.61548
std	1.747595	0.179633	0.196686	1.355917	0.047267	10.25048
min	4.600000	0.120000	0.000000	0.900000	0.012000	1.00000
25%	7.100000	0.392500	0.090000	1.900000	0.070000	7.00000
50%	7.900000	0.520000	0.250000	2.200000	0.079000	13.00000
75%	9.100000	0.640000	0.420000	2.600000	0.090000	21.00000
max	15.900000	1.580000	1.000000	15.500000	0.611000	68.00000

```
sns.countplot(x='quality', data=data)
plt.title("Wine Quality Count")
plt.show()
```




```
plt.figure(figsize=(10,8))
sns.heatmap(data.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```




```
X = data.drop('quality', axis=1)
y = data['quality']
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()

model.fit(X_train, y_train)
```

```
▾ RandomForestClassifier
RandomForestClassifier()
```

```
predictions = model.predict(X_test)

print(predictions)
```

```
[5 6 5 5 6 7 5 5 6 5 7 6 6 6 5 5 6 5 5 7 6 6 5 6 5 5 7 6 5 6 5 6 7 6 5 5
 6 6 6 6 6 6 5 5 5 5 6 5 6 7 5 5 7 6 6 6 6 6 5 5 6 6 6 6 6 6 5 5 6 6 5 5
 5 5 5 6 5 6 6 6 6 5 5 6 5 5 6 6 5 6 5 5 5 5 5 6 5 5 6 7 5 6 5 5 6 6 6 7
 5 5 6 5 5 6 6 6 5 5 5 5 6 5 6 6 6 5 6 5 5 7 6 7 5 5 5 5 6 7 5 5 5 6 5 5
 5 5 6 5 6 5 6 5 5 5 7 6 5 5 6 6 7 5 6 6 6 5 6 6 6 5 5 5 6 6 6 5 6 5 6 5
 5 6 5 6 6 6 6 7 6 6 6 5 6 7 6 6 6 5 5 7 7 5 5 6 6 5 6 5 6 5 6 6 5 7 6 6
 5 5 5 6 6 6 6]
```

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)
```

```
Accuracy: 0.6899563318777293
```

```
from sklearn.metrics import classification_report

print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           4       0.00      0.00      0.00         6
           5       0.72      0.78      0.75        96
           6       0.66      0.70      0.68        99
           7       0.70      0.54      0.61        26
           8       0.00      0.00      0.00         2

    accuracy                           0.69       229
   macro avg       0.42      0.40      0.41       229
weighted avg       0.67      0.69      0.68       229

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.p
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.p
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.p
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
print("Wine Quality Prediction Project Completed Successfully")
```

```
Wine Quality Prediction Project Completed Successfully
```